

Exception Handling in Python (Detailed Guide)

Exception handling is a mechanism that allows a program to deal with runtime errors gracefully instead of crashing abruptly.

Without exception handling, a single error can terminate the entire program.

What is an Exception?

An **exception** is an event that occurs during program execution that interrupts the normal flow of the program.

Examples

```
print(10 / 0)
```

Output:

```
ZeroDivisionError: division by zero
```

```
numbers = [1, 2, 3]  
print(numbers[5])
```

Output:

```
IndexError: list index out of range
```

Why Use Exception Handling?

It helps to:

- Prevent program crashes
 - Display meaningful error messages
 - Recover from errors
 - Separate error-handling logic from normal logic
 - Clean up resources properly
-

The `try-except` Block

Basic syntax:

```
try:
    # Code that may raise an exception
except ExceptionType:
    # Code to handle the exception
```

Example:

```
try:
    num = int(input("Enter a number: "))
    print(100 / num)

except ZeroDivisionError:
    print("You cannot divide by zero.")
```

Flow of Execution

Suppose:

```
try:
    print("Step 1")
    x = 10 / 0
    print("Step 2")

except ZeroDivisionError:
    print("Error handled")

print("Program continues")
```

Output:

```
Step 1
Error handled
Program continues
```

Explanation

1. `try` block starts.
 2. Step 1 executes.
 3. Exception occurs.
 4. Remaining statements inside `try` are skipped.
 5. Matching `except` block executes.
 6. Program continues.
-

Catching Multiple Exceptions

You can have multiple `except` blocks.

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
```

```
except ValueError:
    print("Please enter a valid integer.")

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Example:

Input:

abc

Output:

Please enter a valid integer.

Handling Multiple Exceptions Together

Syntax:

```
except (Exception1, Exception2):
```

Example:

```
try:
    x = int(input())
    print(10 / x)

except (ValueError, ZeroDivisionError):
    print("Invalid input or division by zero.")
```

The Generic Exception

You can catch almost all exceptions using `Exception`.

```
try:
    x = int(input())
    print(10 / x)

except Exception as e:
    print("An error occurred:", e)
```

Example Output:

An error occurred: division by zero

as e

Stores the exception object.

```
print(type(e))
print(e)
```

Example:

```
try:
    int("abc")

except Exception as e:
    print(type(e))
    print(e)
```

Output:

```
<class 'ValueError'>
invalid literal for int() with base 10: 'abc'
```

The `else` Clause

`else` executes only if no exception occurs.

Syntax:

```
try:
    ...
except:
    ...
else:
    ...
```

Example:

```
try:
    x = int(input("Enter number: "))
    result = 100 / x

except ZeroDivisionError:
    print("Division by zero.")

except ValueError:
    print("Invalid input.")

else:
    print("Result:", result)
```

Input:

25

Output:

Result: 4.0

The `finally` Clause

`finally` always executes whether an exception occurs or not.

Syntax:

```
try:
    ...
except:
    ...
finally:
    ...
```

Example:

```
try:
    file = open("sample.txt")
    data = file.read()

except FileNotFoundError:
    print("File not found.")

finally:
    print("Execution completed.")
```

Output:

```
File not found.
Execution completed.
```

Use of `finally`

Commonly used for:

- Closing files
- Closing database connections
- Releasing locks
- Cleaning temporary resources

Example:

```
file = None

try:
    file = open("data.txt")
    print(file.read())

except FileNotFoundError:
    print("File not found.")

finally:
    if file:
        file.close()
```

```
print("File closed.")
```

Complete Structure

```
try:
    # risky code

except ExceptionType1:
    # handle error

except ExceptionType2:
    # handle another error

else:
    # executes if no exception

finally:
    # always executes
```

Example:

```
try:
    num = int(input("Enter number: "))
    result = 50 / num

except ValueError:
    print("Invalid input.")

except ZeroDivisionError:
    print("Cannot divide by zero.")

else:
    print("Answer:", result)

finally:
    print("Thank you.")
```

Raising Exceptions

You can generate exceptions manually using `raise`.

Syntax:

```
raise ExceptionType("message")
```

Example:

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative.")
```

Output:

```
ValueError: Age cannot be negative.
```

Re-raising Exceptions

Sometimes you partially handle an exception and then pass it upward.

Example:

```
try:
    x = int(input())

except ValueError:
    print("Logging the error")
    raise
```

Input:

```
abc
```

Output:

```
Logging the error
ValueError: invalid literal for int() with base 10: 'abc'
```

User-Defined Exceptions

You can create your own exception classes.

Syntax:

```
class MyException(Exception):
    pass
```

Example:

```
class InsufficientBalanceError(Exception):
    pass

balance = 1000
withdraw = 1500

if withdraw > balance:
    raise InsufficientBalanceError("Insufficient balance.")
```

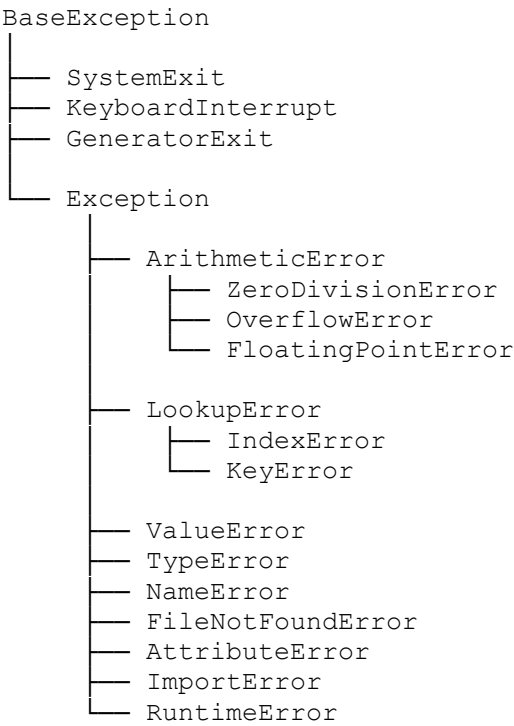
Output:

```
InsufficientBalanceError: Insufficient balance.
```

Exception Hierarchy

Python exceptions follow an inheritance hierarchy.

Simplified hierarchy:



Common Built-in Exceptions

Exception	Cause
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Wrong data type
IndexError	Invalid list index
KeyError	Dictionary key missing
NameError	Variable not defined
FileNotFoundError	File does not exist
AttributeError	Invalid attribute
ImportError	Module import failure
RuntimeError	Generic runtime issue

Best Practices

1. Catch specific exceptions

Good:

```
except ValueError:
    ...
```

Avoid:

```
except:
    ...
```

2. Keep `try` blocks small

Good:

```
try:
    result = 10 / x
except ZeroDivisionError:
    ...
```

Avoid:

```
try:
    # hundreds of lines
except:
    ...
```

3. Use `finally` for cleanup

```
finally:
    connection.close()
```

4. Do not silently ignore exceptions

Avoid:

```
except:
    pass
```

It hides bugs.

5. Create custom exceptions for business logic

Example:

```
class InvalidPasswordError(Exception):  
    pass
```

This makes code more readable and maintainable.

Real-World Example: ATM Withdrawal

```
class InsufficientBalanceError(Exception):  
    pass  
  
balance = 5000  
  
try:  
    amount = int(input("Enter withdrawal amount: "))  
  
    if amount > balance:  
        raise InsufficientBalanceError(  
            "Balance is insufficient."  
        )  
  
    balance -= amount  
    print("Withdrawal successful.")  
    print("Remaining Balance:", balance)  
  
except ValueError:  
    print("Please enter a valid amount.")  
  
except InsufficientBalanceError as e:  
    print(e)  
  
finally:  
    print("Transaction completed.")
```

Summary

Exception handling in Python revolves around these keywords:

- try → Code that may cause an exception
- except → Handle exceptions
- else → Runs if no exception occurs
- finally → Always executes for cleanup
- raise → Manually trigger exceptions
- Custom exceptions → Define application-specific errors

Proper exception handling makes Python programs more robust, user-friendly, and easier to maintain.

Link : [CHATGPT](#) [All](#)